1. Introducción

En esta actividad comenzamos con el desarrollo de un prototipo de una ambulancia, el cual posteriormente tendrá diferentes funciones para poder controlar su movimiento. La idea principal del proyecto es que las llantas del lado izquierdo puedan ser controladas de manera independiente a las del lado derecho, para así poder tener un mayor control sobre la dirección del carrito.

En esta primera etapa nos enfocamos principalmente en las llantas del lado izquierdo, utilizando un Arduino UNO, un controlador de motores L293D, botones y motores. Los botones funcionan como la forma en la que el usuario le indica al carrito qué movimiento realizar, mientras que el Arduino recibe estas señales y manda las instrucciones correspondientes al motor. En este avance se buscó que las llantas pudieran moverse hacia adelante y hacia atrás, dejando para una siguiente parte la implementación completa de los giros.

Con esto se empieza a formar la base del sistema de control de la ambulancia, ya que más adelante se podrán integrar los dos lados de las llantas y hacer que el carrito pueda cambiar de dirección dependiendo de las instrucciones que reciba.

2. Objetivo

El objetivo de esta actividad es desarrollar la primera parte del sistema de movimiento de un prototipo de ambulancia utilizando Arduino. Se busca controlar de manera independiente las llantas de ambos lados mediante botones, logrando que los motores puedan girar hacia adelante, hacia atrás o en direcciones opuestas dependiendo de la instrucción que se envíe.

También se busca comprender cómo utilizar el Arduino junto con el controlador L293D para controlar múltiples motores, además de aprender cómo las entradas de los botones pueden utilizarse para generar diferentes movimientos (avanzar, retroceder y girar). Esta parte servirá como base para posteriormente integrar más funciones de seguridad al carrito.

3. Materiales

Para realizar esta parte del proyecto se utilizaron los siguientes materiales:

- 1 Arduino UNO
- 1 controlador de motores L293D
- 4 motores de corriente directa
- 4 botones pulsadores
- 4 resistencias
- 1 protoboard
- Cables jumper
- Fuente de alimentación
- Computadora para programar el Arduino
- Cable USB para conectar el Arduino a la computadora

4. Metodología

Al iniciar el sistema, el Arduino está programado para revisar de manera constante el estado de los cuatro botones mediante la lectura digital. El usuario interactúa presionando los botones para indicarle al prototipo la acción que debe realizar. Al presionar el botón para avanzar o retroceder, el Arduino envía la señal correspondiente al controlador L293D, el cual establece la dirección y acciona los motores. Si no se detecta ningún botón presionado, se ejecuta la instrucción para apagar los motores.

4.1 Conexión de los botones

Se utilizaron cuatro botones pulsadores y cuatro resistencias, los cuales fueron colocados en la protoboard. Las conexiones hacia el Arduino se distribuyeron en los pines digitales para poder recibir la señal correspondiente a cada movimiento:

- Botón Izquierda: Conectado como entrada al pin 2.
- Botón Atrás: Conectado como entrada al pin 3.
- Botón Adelante: Conectado como entrada al pin 4.
- Botón Derecha: Conectado como entrada al pin 5.

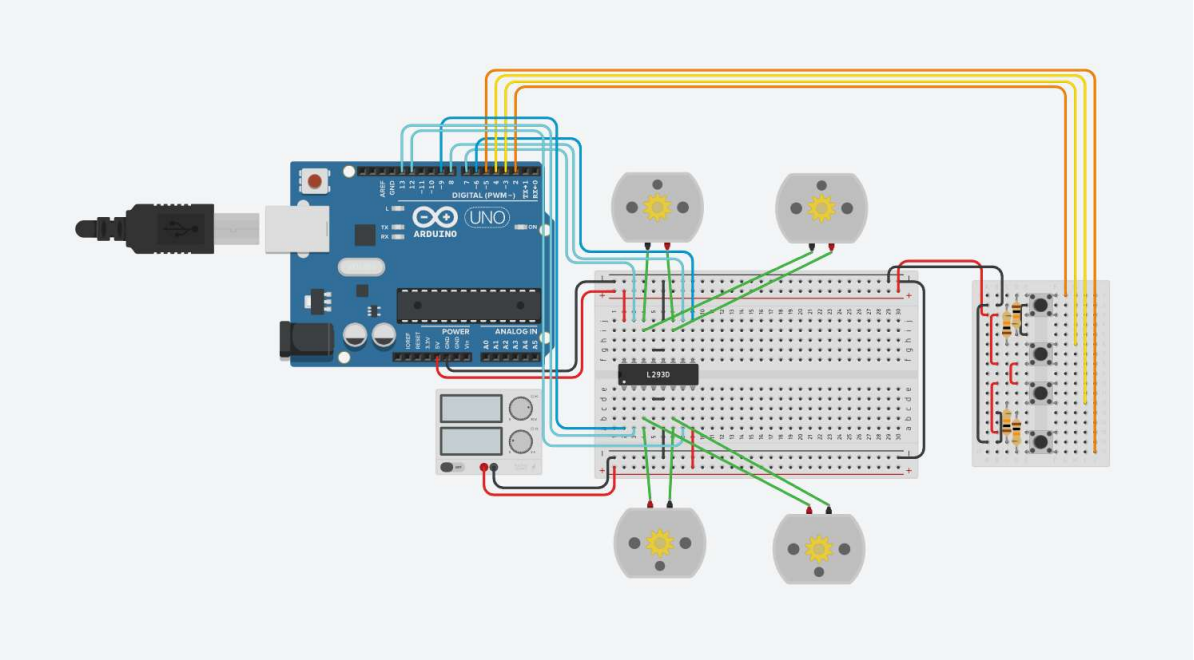
4.2 Funcionamiento y conexión de los motores y el controlador L293D

Los motores de corriente directa (DC) funcionan girando en un sentido u otro dependiendo de la polaridad de la energía que reciben. Para poder controlar este giro desde el Arduino sin tener que cambiar los cables físicamente, se implementó un controlador de motores L293D. Este circuito integrado funciona internamente como un puente H, lo que permite cambiar la dirección en la que fluye la corriente y, por lo tanto, decidir si el motor gira hacia adelante o hacia atrás. Además, el L293D es necesario porque aísla la etapa de control (el Arduino) de la etapa de potencia; los motores requieren más corriente de la que el Arduino puede entregar, por lo que el controlador toma la energía de una fuente de alimentación externa y la dirige a los motores de forma segura.

Para lograr controlar las llantas de ambos lados, los pines del controlador L293D se conectaron de la siguiente manera:

- **Pin Enable (Habilitador 1,2):** Se conectó al puerto digital 6 del Arduino (definido como motorEna) para el lado izquierdo, y al puerto 9 (motorEnb) para el lado derecho. Estos pines sirven para encender o apagar el canal de los motores y permiten controlar la velocidad utilizando modulación por ancho de pulso (PWM).
- **Pines Input 1 e Input 2 (Entradas de control):** Se conectaron a los puertos 7 (motorIn1) y 8 (motorIn2) para el lado izquierdo, y a los puertos 12 (motorIn3) y 13 (motorIn4) para el lado derecho. Estos pines reciben las instrucciones lógicas para decidir el sentido de giro.
- **Pines Output 1 y Output 2 (Salidas al motor):** Estos pines se conectaron directamente a las terminales de los motores de corriente directa de sus respectivos lados. Son los encargados de transmitir el voltaje con la polaridad correcta dictada por las entradas.
- **Pines GND (Tierra):** Se conectaron a la línea negativa de la protoboard para unificar las tierras del Arduino, la fuente de alimentación externa y el controlador L293D, cerrando así el circuito.
- **Pines de voltaje (Vcc1 y Vcc2):** Uno de estos pines se conecta a los 5V del Arduino para la alimentación lógica del chip, y el otro se conecta a la línea positiva de la fuente de alimentación externa, la cual es la encargada de suministrar la energía con mayor potencia directamente a los motores.

4.3 Diagrama de conexión



5. Código utilizado

```
// --- PINES DE LOS BOTONES ---
int botonIzquierda = 2;
int botonAtras = 3;
int botonAdelante = 4;
int botonDerecha = 5;

// --- MOTORES LADO IZQUIERDO (Canal 1) ---
int motorEna = 6; // Control de velocidad Izquierda (PWM)
int motorIn1 = 7; // Control de dirección Izquierda 1
int motorIn2 = 8; // Control de dirección Izquierda 2

// --- MOTORES LADO DERECHO (Canal 2) ---
// Asumiendo pines 9, 10 y 11 para el lado derecho del L293D
int motorEnb = 9;  // Control de velocidad Derecha (PWM)
int motorIn3 = 12; // Control de dirección Derecha 1
int motorIn4 = 13; // Control de dirección Derecha 2

// Velocidad base de los motores (0 a 255)
int velocidad = 255;

```

```
void setup() {
  // Configurar botones
  pinMode(botonIzquierda, INPUT);
  pinMode(botonAtras, INPUT);
  pinMode(botonAdelante, INPUT);
  pinMode(botonDerecha, INPUT);

  // Configurar pines del L293D (Izquierda)
  pinMode(motorEna, OUTPUT);
  pinMode(motorIn1, OUTPUT);
  pinMode(motorIn2, OUTPUT);

  // Configurar pines del L293D (Derecha)
  pinMode(motorEnb, OUTPUT);
  pinMode(motorIn3, OUTPUT);
  pinMode(motorIn4, OUTPUT);

  // Iniciar con todos los motores apagados
  digitalWrite(motorIn1, LOW);
  digitalWrite(motorIn2, LOW);
  analogWrite(motorEna, 0);

  digitalWrite(motorIn3, LOW);
  digitalWrite(motorIn4, LOW);
  analogWrite(motorEnb, 0);
}

```

```
void loop() {
  // 1. Leer el estado de los 4 botones
  bool estadoIzquierda = digitalRead(botonIzquierda);
  bool estadoAtras     = digitalRead(botonAtras);
  bool estadoAdelante  = digitalRead(botonAdelante);
  bool estadoDerecha   = digitalRead(botonDerecha);

  // 2. Evaluar qué botón está presionado y mover el carrito
  if (estadoAdelante == HIGH) {
    // Avanzar: Ambos lados giran hacia adelante
    digitalWrite(motorIn1, HIGH);
    digitalWrite(motorIn2, LOW);
    analogWrite(motorEna, velocidad);

    digitalWrite(motorIn3, HIGH);
    digitalWrite(motorIn4, LOW);
    analogWrite(motorEnb, velocidad);
  }
  else if (estadoAtras == HIGH) {
    // Retroceder: Ambos lados giran hacia atrás
    digitalWrite(motorIn1, LOW);
    digitalWrite(motorIn2, HIGH);
    analogWrite(motorEna, velocidad);

    digitalWrite(motorIn3, LOW);
    digitalWrite(motorIn4, HIGH);
    analogWrite(motorEnb, velocidad);
  }
```

```
  else if (estadoIzquierda == HIGH) {
    // Girar Izquierda: Lado derecho avanza, lado izquierdo retrocede
    digitalWrite(motorIn1, LOW);
    digitalWrite(motorIn2, HIGH);
    analogWrite(motorEna, velocidad);

    digitalWrite(motorIn3, HIGH);
    digitalWrite(motorIn4, LOW);
    analogWrite(motorEnb, velocidad);
  }
  else if (estadoDerecha == HIGH) {
    // Girar Derecha: Lado izquierdo avanza, lado derecho retrocede
    digitalWrite(motorIn1, HIGH);
    digitalWrite(motorIn2, LOW);
    analogWrite(motorEna, velocidad);

    digitalWrite(motorIn3, LOW);
    digitalWrite(motorIn4, HIGH);
    analogWrite(motorEnb, velocidad);
  }
  else {
    // Si no hay ningún botón presionado, apagar todos los motores
    digitalWrite(motorIn1, LOW);
    digitalWrite(motorIn2, LOW);
    analogWrite(motorEna, 0);
```

```

    digitalWrite(motorIn3, LOW);
    digitalWrite(motorIn4, LOW);
    analogWrite(motorEnb, 0);
  }

  // Pequeña pausa para estabilizar la lectura de los botones
  delay(20);
}
```

6. Explicación del código

El código en Arduino comúnmente se divide en dos secciones, "setup" y "loop". En la sección setup, se ponen las instrucciones que se desean ejecutar únicamente una vez cuando el sistema inicie, y en loop, el que se desea repetir mientras el Arduino esté encendido.

Lo primero que se hizo fue crear cuatro variables para los botones (2,3,4 y 5), nombradas de acuerdo a la dirección: "botonIzquierda", "botonAtras", "botonAdelante" y "botonDerecha". También se crearon 6 variables las cuales cumplen con la función de controlar los motores de ambos lados mediante el L293D. Estas variables fueron "motorEna" y "motorEnb" para controlar la velocidad, así como "motorIn1", "motorIn2", "motorIn3" y "motorIn4" que deciden hacia qué sentido gira cada lado. La última variable creada fue "velocidad", a la cual se le asignó el valor 255.

Dentro de la sección setup, primeramente utilizamos la función pinMode para hacer que los cuatro botones sean entradas utilizando INPUT. Algo similar se hizo con los pines de los motores, los cuales fueron configurados para ser salidas utilizando OUTPUT. Para finalizar esta sección, se configuraron los motores en un estado

LOW y velocidad en 0, para que al iniciar el programa el carrito no se mueva a menos de que reciba una instrucción.

Para la sección loop, primeramente se colocó la función digitalRead con el fin de revisar constantemente qué está pasando con los cuatro botones, guardando su estado en nuevas variables.

Después, se utiliza una serie de condiciones if y else if con las que se determina el movimiento del motor, asignando las lógicas de giro y aplicando la velocidad máxima con analogWrite:

- Si el botón presionado es el de movimiento hacia adelante, se ejecuta un estado HIGH para que ambos lados avancen.
- Si el botón presionado es el de hacia atrás, se invierten los estados para que ambos lados retrocedan.
- Si el botón presionado es Izquierda, se configura el motor izquierdo en reversa y el derecho hacia adelante, permitiendo girar sobre su propio eje.
- Si el botón presionado es Derecha, se configura el motor derecho en reversa y el izquierdo hacia adelante.
- Si ningún botón es presionado, se ejecuta un bloque else que manda todos los pines a LOW, apagando el carrito.

Para finalizar con esta sección, se usó delay(20);, una función que hará una pausa de 20 milisegundos que evitará que se detecten varias pulsaciones del botón dentro de una sola.

7. Resultados

Después de cargar el código y realizar conexiones, se realizaron distintas pruebas con el fin de comprobar si el prototipo funciona como se espera. Con ellas, pudo comprobarse que al presionar los botones, el carrito logra trasladarse y cambiar de dirección exitosamente, demostrando que el Arduino detecta los botones correctamente y envía las señales de control de los dos canales (izquierdo y derecho) al L293D.

Botón presionado	Resultado
Ninguno	El motor no se mueve
Adelante	Los cuatro motores giran hacia adelante
Atrás	Los cuatro motores giran hacia atrás
Izquierda	El lado derecho avanza, el lado izquierdo retrocede (giro a la izquierda)
Derecha	El lado izquierdo avanza, el lado derecho retrocede (giro a la derecha).

8. Conclusión

En este avance logramos crear un sistema de control completo para la tracción de un prototipo de ambulancia. Para la realización de esto, se usaron distintas herramientas como un Arduino, cuatro botones, motores y un controlador L293D.

Con el programa pudimos controlar el giro de los motores de ambos lados usando los botones como medio, con lo que se pudo lograr movimiento hacia adelante, hacia atrás y giros sobre su propio eje dependiendo de lo que el usuario eligiera. Con las pruebas pudimos comprobar que el sistema funcionaba de una manera correcta, respondiendo bien a las pulsaciones y logrando una maniobrabilidad total, con lo cual se pudo cumplir el objetivo de esta actividad.

9. Referencias

SparkFun Electronics. (s.f.). *Pull-up resistors*. SparkFun Learn. Recuperado el 28 de septiembre de 2026, de https://learn.sparkfun.com/tutorials/pull-up-resistors/all

The Engineering Projects. (2017, julio). *Introduction to L293D*. Recuperado el 28 de septiembre de 2026, de https://www.theengineeringprojects.com/2017/07/introduction-to-l293d.html